

Open Geospatial Consortium

Submission Date: 2024-10-03

Approval Date: 2025-09-09

Publication Date: 2025-10-14

External identifier of this OGC® document: <http://www.opengis.net/doc/CS/stac/1.1>

Internal reference number of this OGC® document: 25-004

Version: 1.1

Category: OGC® Community Standard

Editors: STAC Community

SpatioTemporal Asset Catalog (STAC) Community Standard

Copyright notice

Copyright © 2025 Open Geospatial Consortium

To obtain additional rights of use, visit <https://www.ogc.org/legal/>

Warning

This document is an OGC Member endorsed international Community standard. This Community standard was developed outside of the OGC and the originating party may continue to update their work; however, this document is fixed in content. This document is available on a royalty free, non-discriminatory basis. Recipients of this document are invited to submit, with their comments, notification of any relevant patent rights of which they are aware and to provide supporting documentation.

Document type: OGC® Community Standard

Document subtype:

Document stage: Approved

Document language: English

STAC Community

The organization listed above has granted the Open Geospatial Consortium (OGC) a nonexclusive, royalty-free, paid up, worldwide license to copy and distribute this document and to modify this document and distribute copies of the modified version under a Creative Commons Attribution 4.0 International License (see below).

License Agreement

The Standard is licensed under the Creative Commons Attribution 4.0 International License (the “License”). You are directed to the License for specific details at: <https://creativecommons.org/licenses/by/4.0/deed.en>.

Source

The original source for the STAC specification is located on GitHub here: <https://github.com/radiantearth/stac-spec/tree/v1.1.0>



SpatioTemporal Asset Catalog



About

The SpatioTemporal Asset Catalog (STAC) family of specifications aim to standardize the way geospatial asset metadata is structured and queried. A “spatiotemporal asset” is any file that represents information about the Earth at a certain place and time. The original focus was on scenes of satellite imagery, but the specifications now cover a broad variety of uses, including sources such as aircraft and drone and data such as hyperspectral optical, synthetic aperture radar (SAR), video, point clouds, lidar, digital elevation models (DEM), vector, machine learning labels, and composites like NDVI and mosaics. STAC is intentionally designed with a minimal core and flexible extension mechanism to support a broad set of use cases. This specification has matured over the past several years, and is used in [numerous production deployments](#).

This is advantageous to providers of geospatial data, as they can simply use a well-designed, standard format and API without needing to design their own proprietary one. This is advantageous to consumers of geospatial data, as they can use existing libraries and tools to access metadata, instead of needing to write new code to interact with each data provider’s proprietary formats and APIs.

The STAC specifications define related JSON object types connected by link relations to support a [HATEOAS](#) -style traversable interface and a [RESTful](#) API providing additional browse and search interfaces. Typically, several STAC specifications are composed together to create an implementation. The

Item, **Catalog**, and **Collection** specifications define a minimal core of the most frequently used JSON object types. Because of the hierarchical structure between these objects, a STAC catalog can be implemented in a completely ‘static’ manner as a group of hyperlinked Catalog, Collection, and Item URLs, enabling data publishers to expose their data as a browsable set of files. If more complex query abilities are desired, such as spatial or temporal predicates, the [STAC API specification](#) can be implemented as a web service interface to query over a group of STAC objects, usually held in a database.

To the greatest extent possible, STAC uses and extends existing specifications. The most important object in STAC is an **Item**, which is simply a [GeoJSON Feature](#) with a well-defined set of additional attributes (“foreign members”). The **STAC API** extends the [OGC API - Features - Part 1: Core](#) with additional web service endpoints and object attributes.

Current version and branches

The [master branch](#) is the ‘stable’ version of the spec. It is currently version **1.1.0** of the specification. The STAC specification follows [Semantic Versioning](#), so any breaking change will require the spec to go to 2.0.0.

The [dev](#) branch is where active development takes place, and may have inconsistent examples. Whenever dev stabilizes, a release is cut and we merge dev in to master. So master should be stable at any given time. More information on how the STAC development process works can be found in [process.md](#).

Communication

Our [gitter channel](#) is the best place to ask questions or provide feedback. The majority of communication about the evolution of the specification takes place in the [issue tracker](#) and in [pull requests](#).

In this Specification

The **Item**, **Catalog**, **Collection**, and **STAC API** specifications are intended to be used together, but are designed so each piece is small, self-contained, and reusable in other contexts.

- [Overview](#) describes the three core object type specifications and how they relate to one another.
- [Item Specification](#) defines a STAC **Item**, which is a [GeoJSON Feature](#) with additional fields (“foreign members”) for attributes like time and links to related entities and assets (including thumbnails). This is the core entity that describes the data to be discovered.
- [Catalog Specification](#) specifies a structure to link various STAC Items together to be crawled or browsed. It is a simple, flexible JSON file of links to Items, Catalogs or Collections that can be used in a variety of ways.
- [Collection Specification](#) provides additional information about a spatio-temporal collection of data. In the context of STAC it is most likely a related group of STAC Items that is made available by a data provider. It includes things like the spatial and temporal extent of the data, the license, keywords, etc. It enables discovery at a higher level than individual Item objects, providing a simple way to describe sets of data.
- [Commons](#) describes parts of the specification that are shared across the specifications listed above. This includes [assets](#), [links](#) and [common metadata](#).
- [Examples](#): The [examples/](#) folder contains examples for all three specifications, linked together to form two complete examples. Each spec and extension links in to highlight particular files that demonstrate key concepts.
- [Extensions](#) describe how STAC can use extensions that extend the functionality of the core spec or

add fields for specific domains. Extensions can be published anywhere, although the preferred location for public extensions is in the [GitHub stac-extensions organization](#) .

- **Additional documents:** The supporting documents include a complementary [best practices](#) document, and information on contributing (links in the next section). We also maintain a [changelog](#) of what was modified in each version.

Contributing

Anyone building software that catalogs imagery or other geospatial assets is welcome to collaborate. Beforehand, please review our [guidelines for contributions](#) and [code of conduct](#) . You may also be interested in our overall [process](#) , and the [principles](#) that guide our collaboration

STAC Overview

There are three component specifications that together make up the core SpatioTemporal Asset Catalog specification. Each can be used alone, but they work best in concert with one another. The [STAC API specification](#) builds on top of that core, but is out of scope for this overview. An [Item](#) represents a single [spatiotemporal asset](#) as [GeoJSON](#) so it can be searched. The [Catalog](#) specification provides structural elements, to group Items and [Collections](#) . Collections *are* catalogs, that add more required metadata and describe a group of related Items. For more on the differences see the [section below](#).

A [UML diagram](#) of the [STAC model](#) is also provided to help with navigating the specification.

Foundations

STAC is built on top of many great standards and practices. Every part of STAC is [JSON](#) , and [GeoJSON](#) provides the core geometry fields and [features](#) definition. All fields are described in the specifications, and the acceptable values are defined with [JSON Schema](#) . The released JSON Schemas provide the core testing definitions, and are used in an array of validation tools. We also rely on [RFC 8288 \(Web Linking\)](#) to express relationships between resources, and IANA [Media Types](#) to describe file formats and format contents. The [OGC API - Features](#) standard is a final core building block. The STAC Collection extends the [Collection](#) JSON defined in OGC API - Features (and the full API definition is the foundation for the STAC API specification).

The STAC specifications are written to be understandable without needing a full background in these. But if you want to get deep into STAC tool implementation and are not familiar with any of the standards mentioned above it is recommended to read up on them. STAC development is guided by set of core philosophical tenets, like building small reusable parts that are loosely coupled, focusing on developers, and more - see our the [principles](#) document to learn more.

Note: Setting a field in JSON to `null` is not equivalent to a field not appearing in STAC, as JSON Schema tools treat them differently. STAC defines `null` explicitly for some fields, where it has a particular meaning. So `null` should not be used unless the STAC spec defines its use - instead the field should be left out entirely.

Item Overview

Fundamental to any SpatioTemporal Asset Catalog, an [Item](#) object represents a unit of data and metadata, typically representing a single scene of data at one place and time. A STAC Item is a [GeoJSON](#) [Feature](#)

and can be easily read by any modern GIS or geospatial library, and it describes a [SpatioTemporal Asset](#). The STAC Item JSON specification uses the GeoJSON geometry to describe the location of the asset, and then includes additional information:

- the time the asset represents;
- a thumbnail for quick browsing;
- asset links, to enable direct download or streaming access of the asset;
- relationship links, allowing users to traverse other related resources and STAC Items.

A STAC Item can contain additional fields and JSON structures to communicate more information about the asset, so it can be easily searched. STAC provides a core set of [Common Metadata](#) and there is a wider community working on a variety of [STAC Extensions](#) that provide shared metadata for more specific domains. Both aim to describe data with well known, well defined terms to enable consistent publishing and better search. For more recommendations on selecting fields for an Item see [this section](#) of the best practices document.

What is a SpatioTemporal Asset

A ‘spatiotemporal asset’ is any file that represents information about the earth captured in a certain space and time. Examples include Imagery (from satellites, planes and drones), SAR, Point Clouds (from LiDAR, Structure from Motion, etc), Data Cubes, Full Motion Video, and data derived from any of those. The key is that the GeoJSON is not the actual ‘thing’, but instead references files and serves as an index to the ‘assets’. It is [not recommended](#) to use STAC to refer to traditional vector data layers (shapefile, geopackage) as assets, as they don’t quite fit conceptually.

Catalogs vs Collections

Before we go deep into the Catalogs and Collections, it is worth explaining the relationship between the two and when you might want to use one or the other.

A Catalog is a very simple construct - it just provides links to Items or to other Catalogs. The closest analog is a folder in a file structure, it is the container for Items, but it can also hold other containers (folders / catalogs).

The Collection entity shares most fields with the Catalog entity but has a number of additional fields: license, extent (spatial and temporal), providers, keywords and summaries. Every Item in a Collection links back to their Collection, so clients can easily find fields like the license. Thus every Item implicitly shares the fields described in their parent Collection. Collection entities can be used just like Catalog entities to provide structure, as they provide all the same options for linking and organizing.

But what *should* go in a Collection, versus just in a Catalog? A Collection will generally consist of a set of assets that are defined with the same properties and share higher level metadata. In the satellite world these would typically all come from the same sensor or constellation. It corresponds directly to what others call a “dataset series” (ESA, ISO 19115), “collection” (CNES, NASA), and “dataset” (JAXA, DCAT). So if all your Items have the same properties, they probably belong in the same Collection. But the construct is deliberately flexible, as there may be good reasons to break the recommendation.

Catalogs in turn are used for two main things:

- Split overly large collections into groups
- Group collections into a catalog of Collections (e.g. as entry point for navigation to several

Collections).

The first case allows users to browse down into the Items of large collections. A collection like Landsat usually would start with path and row Catalogs to group by geography, and then year, month and day groups to enable deeper grouping. [Dynamic catalogs](#) can provide multiple grouping paths, serving as a sort of faceted search.

The second case is used when one wants to represent diverse data in a single place. If an organization has an internal catalog with Landsat 8, Sentinel 2, NAIP data and several commercial imagery providers then they'd have a root Catalog that would link to a number of different Collections.

So in conclusion it's best to use Collections for what you want users to find as the starting point, and then Catalogs are just for structuring and grouping the data. Future work includes a mechanism to actually search Collection-level data, hopefully in concert with other specifications.

Catalog Overview

NOTE: The below examples all say Catalog, but those can all be Collections as well, as it has all the fields necessary to serve as a Catalog

There are two required element types of a Catalog: Catalog and Item. A STAC Catalog points to [STAC Items](#), or to other STAC catalogs. It provides a simple linking structure that can be used recursively so that many Items can be included in a single Catalog, organized however the implementor desires.

STAC makes no formal distinction between a “root” Catalog and the “child” Catalogs. A root Catalog is simply the top-most Catalog or Collection – it has no parent. A nested catalog structure is useful (and recommended) for breaking up massive numbers of catalog Items into logical groupings. For example, it might make sense to organize a catalog by date (year, month, day), or geography (continent, country, state/prov). See the [Catalog Layout](#) best practices section for more.

A simple STAC structure might look like this:

- catalog (root)
 - catalog
 - catalog
 - item
 - asset
 - item
 - asset
 - item
 - asset
 - asset

This example might be considered a somewhat “typical” structure. However, Catalogs and Items can describe a number of different relationships. The following shows various relationships between catalogs and items:

- Catalog -> Item (this is a common structure for a catalog to list links to Items)
- Catalog -> Catalog (this is a common tree structure to group sets of Items. Each catalog in this relationship may also include Item links as well as catalog links)

The relationships are all described by a common `links` object structure, making use of the `rel` field to further describe the relationship.

There are a few types of catalogs that implementors occasionally refer to. These get defined by the `links` structure.

- A **sub-catalog** is a Catalog that is linked to from another Catalog that is used to better organize data. For example a Landsat collection might have sub-catalogs for each Path and Row, so as to create a nice tree structure for users to follow.
- A **root catalog** is a Catalog that only links to sub-catalogs. These are typically entry points for browsing data. Often they will contain the [STAC Collection](#) definition, but in implementations that publish diverse information it may contain sub-catalogs that provide a variety of Collections.
- A **parent catalog** is the Catalog that sits directly above a sub-catalog. Following parent catalog links continuously will naturally end up at a root catalog definition.

It should be noted that a Catalog does not have to link back to all the other Catalogs that point to it. Thus a published root catalog might be a sub-catalog of someone else's structure. The goal is for data providers to publish all the information and links they want to, while also encouraging a natural web of information to arise as Catalogs and Items are linked to across the web.

Static and Dynamic Catalogs

The Catalog specification is designed so it can be implemented as easily as possible. This can be as simple as simply putting linked json files on a file server or an object storage service (like [AWS S3](#)), or it can be generated on the fly by a live server. The first type of implementation is often called a 'static catalog', and any catalog that is not just files is called a 'dynamic catalog'. You can read more about the two types along with recommendations in [this section](#) of the best practices document, along with how to keep a [dynamic catalog in sync](#) with a static one.

Catalog Best Practices

In addition to information about different catalog types, the [best practices document](#) has a number of suggestions on how to organize and implement good catalogs. The [catalog specification](#) is designed for maximum flexibility, so none of these are required, but they provide guidance for implementors who want to follow what most of the STAC community is doing.

- [Catalog Layout](#) is likely the most important section, as following its recommendations will enable catalogs to work better with client tooling that optimizes for known layouts.
- [Use of Links](#) articulates practices for making catalogs that are portable (with relative links through out) and ones that are published in stable locations (with absolute self links).
- [Versioning for Catalogs](#) explains how to use STAC's structure to keep a history of changes made to Items and catalogs.
- [STAC on the Web](#) explains how catalogs should have html versions for each Item and Catalog, as well as ways to achieve that.

Collection Overview

A STAC Collection includes the core fields of the Catalog entity and also provides additional metadata to describe the set of Items it contains. The required fields are fairly minimal - it includes the 4 required Catalog fields (id, description, stac_version and links), and adds license and extents. But there are a number of other common fields defined in the spec, and more common fields are also defined in [STAC extensions](#) . These serve as basic metadata, and ideally Collections also link to fuller metadata (ISO 19115, etc) when it is available.

As Collections contain all of Catalogs' core fields, they can be used just as flexibly. They can have both parent Catalogs and Collections as well as child Items, Catalogs and Collections. Items are strongly recommended to have a link to the Collection they are a part of. Items can only belong to one Collection, so if an Item is in a Collection that is the child of another Collection, then it must pick which one to refer to. Generally the 'closer' Collection, the more specific one, should be the one linked to.

The Collection specification is used standalone quite easily - it is used to describe an aggregation of data, and doesn't require links down to sub-catalogs and Items. This is most often used when the software does operations at the layer / coverage level, letting users manipulate a whole collection of assets at once. They often have an optimized internal format that doesn't make sense to expose as Items. [OpenEO](#) and [Google Earth Engine](#) are two examples that only use STAC collections, and both would be hard-pressed to expose individual Items due to their architectures. For others implementing STAC Collections can also be a nice way to start and achieve some level of interoperability.

STAC Item Specification

- [Overview](#)
- [Item fields](#)
 - [stac_version](#)
 - [stac_extensions](#)
 - [id](#)
 - [geometry](#)
 - [bbox](#)
 - [collection](#)
 - [properties](#)
 - [Properties Object](#)
 - [datetime](#)
 - [Additional Fields](#)
 - [Links](#)
 - [Relation types](#)
 - [Collections](#)
 - [Assets](#)
- [Media Type for STAC Item](#)
- [Extensions](#)

Overview

This document explains the structure and content of a SpatioTemporal Asset Catalog (STAC) Item. An **Item** is a [GeoJSON Feature](#) augmented with [foreign members](#) relevant to a STAC object. These include fields that identify the time range and assets of the Item. An Item is the core object in a STAC Catalog, containing the core metadata that enables any client to search or crawl online catalogs of spatial 'assets' (e.g., satellite imagery, derived data, DEMs).

The same Item definition is used in both [STAC Catalogs](#) and the [Item-related API endpoints](#). Catalogs are simply sets of Items that are linked online, generally served by simple web servers and used for crawling data. The search endpoint enables dynamic queries, for example selecting all Items in Hawaii on June 3, 2015, but the results they return are FeatureCollections of Items.

Items are represented in JSON format and are very flexible. Any JSON object that contains all the required fields is a valid STAC Item.

- Examples:
 - See the [minimal example](#) , as well as a [more fleshed example](#) that contains a number of current best practices.
 - Real world [implementations](#) are also available.
- [JSON Schema](#)

Item fields

This object describes a STAC Item. The fields `id`, `type`, `bbox`, `geometry` and `properties` are inherited from GeoJSON.

Field Name	Type	Description
<code>type</code>	string	REQUIRED. Type of the GeoJSON Object. MUST be set to <code>Feature</code> .
<code>stac_version</code>	string	REQUIRED. The STAC version the Item implements.
<code>stac_extensions</code>	[string]	A list of extensions the Item implements.
<code>id</code>	string	REQUIRED. Provider identifier. The ID should be unique within the Collection that contains the Item.
<code>geometry</code>	GeoJSON Geometry Object null	REQUIRED. Defines the full footprint of the asset represented by this item, formatted according to RFC 7946, section 3.1 if a geometry is provided or section 3.2 if no geometry is provided. REQUIRED if geometry is not null, prohibited if geometry is null.
<code>bbox</code>	[number]	Bounding Box of the asset represented by this Item, formatted according to RFC 7946, section 5 .
<code>properties</code>	Properties Object	REQUIRED. A dictionary of additional metadata for the Item.
<code>links</code>	[Link Object]	REQUIRED. List of link objects to resources and related URLs. See the best practices for details on when the use <code>self</code> links is strongly recommended.
<code>assets</code>	Map<string, Asset Object >	REQUIRED. Dictionary of asset objects that can be downloaded, each with a unique key.
<code>collection</code>	string	The <code>id</code> of the STAC Collection this Item references to. This field is required if a link with a <code>collection</code> relation type is present and is not allowed otherwise.

stac_version

In general, STAC versions can be mixed, but please keep the [recommended best practices](#) in mind.

stac_extensions

A list of extensions the Item implements. The list consists of URLs to JSON Schema files that can be used for validation. This list must only contain extensions that extend the Item specification itself, see the ‘Scope’ for each of the extensions.

id

It is important that an Item identifier is unique within a Collection, and that the [Collection identifier](#) in turn is unique globally. Then the two can be combined to give a globally unique identifier. Items are [strongly](#)

[recommended](#) to have Collections, and not having one makes it more difficult to be used in the wider STAC ecosystem. If an Item does not have a Collection, then the Item identifier should be unique within its root Catalog or root Collection.

As most geospatial assets are already uniquely defined by some identification scheme from the data provider it is recommended to simply use that ID. Data providers are advised to include sufficient information to make their IDs globally unique, including things like unique satellite IDs. See the [id section of best practices](#) for additional recommendations.

geometry

Defines the full footprint of the asset represented by this item, formatted according to RFC 7946.

If a **geometry** is provided, the value must be a Geometry Object according to [RFC 7946, section 3.1](#) with the exception that the type GeometryCollection is not allowed in STAC. If **no geometry** is provided, the value must be `null` according to [RFC 7946, section 3.2](#).

Coordinates are specified in Longitude/Latitude or Longitude/Latitude/Elevation based on [WGS 84](#).

bbox

Bounding Box of the asset represented by this Item using either 2D or 3D geometries, formatted according to [RFC 7946, section 5](#). The length of the array must be $2 \times n$ where n is the number of dimensions. The array contains all axes of the southwesterly most extent followed by all axes of the northeasterly most extent specified in Longitude/Latitude or Longitude/Latitude/Elevation based on [WGS 84](#). When using 3D geometries, the elevation of the southwesterly most extent is the minimum depth/height in meters and the elevation of the northeasterly most extent is the maximum. This field enables more naive clients to easily index and search geospatially. STAC compliant APIs are required to compute intersection operations with the Item's geometry field, not its bbox.

collection

The `id` of the STAC Collection this Item references to with the [collection relation type](#) in the `links` array.

This field provides an easy way for a user to search for any Items that belong in a specified Collection. If present, must be a non-empty string.

properties

Properties Object

Additional metadata fields can be added to the GeoJSON Object Properties. The only required field is `datetime` but it is recommended to add more fields, see [Additional Fields](#) resources below.

Field Name	Type	Description
datetime	string null	REQUIRED. The searchable date and time of the assets, which must be in UTC. It is formatted according to RFC 3339, section 5.6 . <code>null</code> is allowed, but requires <code>start_datetime</code> and <code>end_datetime</code> from common metadata to be set.

datetime

This is likely the acquisition (in the case of single camera type captures) or the ‘nominal’ or representative time in the case of assets that are combined together. Though time can be a complex thing to capture, for this purpose keep in mind the STAC spec is primarily searching for data, so use whatever single date and time is most useful for a user to search for. STAC content extensions may further specify the meaning of the main datetime field, and many will also add more datetime fields. **All times in STAC metadata should be in [Coordinated Universal Time](#) (UTC).** If there’s clearly no meaningful single ‘nominal’ time, it is allowed to use `null` instead. In this case it is **required** to specify a temporal interval with the fields `start_datetime` and `end_datetime` from [common metadata](#) . For example, if your data is a time-series that covers 100 years, it’s not very meaningful to set the datetime to a single timestamp as it would not be found in most searches that searches for a decade of data in that period although the Item actually covers the decade. See [datetime selection](#) in the best practices document for more information.

Additional Fields

Providers should include metadata fields that are relevant for users of STAC, but it is recommended to [select only those necessary for search](#) . Where possible metadata fields should be mapped to the STAC Common Metadata and widely used extensions, to enable cross-catalog search on known fields.

- [STAC Common Metadata](#) - A list of fields commonly used throughout all domains. These optional fields are included for STAC Items by default.
- [Extensions](#) - Additional fields that are more specific, such as [EO](#) , [View](#) .
- [Custom Extensions](#) - It is generally allowed to add custom fields but it is recommended to add multiple fields for related values instead of a nested object, e.g., two fields `view:azimuth` and `view:off_nadir` instead of a field `view` with an object value containing the two fields. The convention (as used within Extensions) is for related fields to use a common prefix on the field names to group them, e.g. `view`. A nested data structure should only be used when the data itself is nested, as with `bands`.

Links

Each link in the `links` array must be a [Link Object](#) .

Relation types

All [common relation types](#) except for `item` can be used in Items. A `self` and `collection` links are **STRONGLY RECOMMENDED**. A link with this `rel` type is *required* for STAC item if the `collection` field in `properties` is present.

[!NOTE] Dynamic catalogs can implement multiple parents through a dynamic browsing interface as they could dynamically create the parent link based on the desired browsing structure (though only 1 parent at a time). Multiple parents are allowed for other types than `application/json`.

Collections

Items are *strongly recommended* to provide a link to a STAC Collection definition. It is important as Collections provide additional information about a set of items, for example the license, provider and other information giving context on the overall set of data that an individual Item is a part of.

If Items are part of a STAC Collection, the [STAC Collection spec requires Items to link back to the Collection](#) . Linking back must happen in two places:

1. The field `collection` in an Item must be filled (see section ‘Item fields’). It is the `id` of a STAC Collection.
2. An Item must also provide a link to the STAC Collection using the [collection relation type](#):

```
"links": [  
  { "rel": "collection", "href": "link/to/collection/record.json" }  
]
```

Multiple collections can point to an Item, but an Item can only point back to a single collection.

Assets

The property `assets` is a dictionary of [Asset Objects](#) , each with a unique key. Each asset refers to data associated with the Item that can be downloaded or streamed. This construct is further detailed in the [Assets](#) document.

Assets in a STAC Item should include the main asset, as well as any ‘sidecar’ files that are related and help a client make sense of the data. Examples of this include extended metadata (in XML, JSON, etc.), unusable data masks, satellite ephemeris data, etc. Some assets (like Landsat data) are represented by multiple files - all should be linked to. It is generally recommended that different processing levels or formats are not exhaustively listed in an Item, but instead are represented by related Items that are linked to, but the best practices around this are still emerging.

It is STRONGLY RECOMMENDED to add to each STAC Item

- a thumbnail with the role `thumbnail` for preview purposes
- one or more data files with the role `data`

Media Type for STAC Item

A STAC Item is a GeoJSON file ([RFC 7946](#)), and thus should use the [application/geo+json](#) as the [Media Type](#) (previously known as the MIME Type).

Extensions

STAC Items are [extensible](#) . Please refer to the [extensions overview](#) to find relevant extensions for STAC Items.

STAC Catalog Specification

- [Catalog fields](#)
 - [stac_version](#)
 - [stac_extensions](#)
 - [links](#)
 - [Relation types](#)
- [Media Type for STAC Catalogs](#)

- [Extensions](#)

This document explains the structure and content of a STAC **Catalog** object. A STAC Catalog object represents a logical group of other Catalog, [Collection](#), and [Item](#) objects. These Items can be linked to directly from a Catalog, or the Catalog can link to other Catalogs (often called sub-catalogs) that contain links to Collections and Items. The division of sub-catalogs is up to the implementor, but is generally done to aid the ease of online browsing by people.

A Catalog object will typically be the entry point into a STAC catalog. Their purpose is discovery: to be browsed by people or be crawled by clients to build a searchable index.

Any JSON object that contains all the required fields is a valid STAC Catalog object.

- [Examples](#)
 - See an example [catalog.json](#). The [collection.json](#) is also a valid Catalog file, demonstrating linking to items (it is also a Collection, so has additional fields)
- [JSON Schema](#)

The [Catalog section of the Overview](#) document provides background information on the structure of Catalogs as well as links to best practices. This specification lays out the requirements and fields to be compliant.

This Catalog specification primarily defines a structure for information to be discoverable. Any use that is publishing a set of related spatiotemporal assets is strongly recommended to also use the STAC Collection specification to provide additional information about the set of Items contained in a Catalog, in order to give contextual information to aid in discovery. STAC Collections all have the same fields as STAC Catalogs, but with different allowed values for type and stac_extensions.

Catalog fields

Element	Type	Description
type	string	REQUIRED. Set to <code>Catalog</code> if this Catalog only implements the Catalog spec.
stac_version	string	REQUIRED. The STAC version the Catalog implements.
stac_extensions	[string]	A list of extension identifiers the Catalog implements.
id	string	REQUIRED. Identifier for the Catalog.
title	string	A short descriptive one-line title for the Catalog.
description	string	REQUIRED. Detailed multi-line description to fully explain the Catalog. CommonMark 0.29 syntax MAY be used for rich text representation.
links	[Link Object]	REQUIRED. A list of references to other documents.

stac_version

In general, STAC versions can be mixed, but please keep the [recommended best practices](#) in mind.

stac_extensions

A list of extensions the Catalog implements. The list consists of URLs to JSON Schema files that can be used for validation. This list must only contain extensions that extend the Catalog specification itself, see the

‘Scope’ for each of the extensions. This must **not** declare the extensions that are only implemented in child Collection objects or child Item objects.

links

Each link in the `links` array must be a [Link Object](#) .

Relation types

All the [common relation types](#) can be used in catalog. A `self` and a `root` links are **STRONGLY RECOMMENDED**. Non-root Catalogs **SHOULD** include a parent link to their parent.

[!NOTE] A link to at least one item or child (Catalog or Collection) is **RECOMMENDED**, but empty catalogs are allowed if there is an intent to populate it or its children were removed.

Media Type for STAC Catalogs

A STAC Catalog is a JSON file ([RFC 8259](#)), and thus should use the [application/json](#) as the [Media Type](#) (previously known as the MIME Type).

Extensions

STAC Catalogs are [extensible](#) . Please refer to the [extensions overview](#) to find relevant extensions for STAC Catalogs.

STAC Collection Specification

- [Overview](#)
- [Collection fields](#)
 - [stac_version](#)
 - [stac_extensions](#)
 - [id](#)
 - [license](#)
 - [providers](#)
 - [Provider Object](#)
 - [extents](#)
 - [Extent Object](#)
 - [Spatial Extent Object](#)
 - [Temporal Extent Object](#)
 - [summaries](#)
 - [Range Object](#)
 - [JSON Schema Object](#)
 - [links](#)
 - [Relation types](#)
 - [assets](#)
 - [item_assets](#)
 - [Item Asset Definition Object](#)
- [Media Type for STAC Collections](#)

- [Standalone Collections](#)
- [Extensions](#)

Overview

The STAC Collection Specification defines a set of common fields to describe a group of Items that share properties and metadata. The Collection Specification shares all fields with the STAC [Catalog Specification](#) (with different allowed values for type and stac_extensions) and adds fields to describe the whole dataset and the included set of Items. Collections can have both parent Catalogs and Collections and child Items, Catalogs and Collections.

A STAC Collection is represented in JSON format. Any JSON object that contains all the required fields is a valid STAC Collection and also a valid STAC Catalog.

STAC Collections are compatible with the [Collection](#) JSON specified in [OGC API - Features](#) , but they are extended with additional fields.

- [Examples](#) :
 - Sentinel 2: A basic standalone example of a [Collection](#) without Items.
 - Simple Example: A [Collection](#) that links to 3 example Items.
 - Extension Collection: An additional [Collection](#) , which is used to highlight various [extension](#) functionality, but serves as another example.
- [JSON Schema](#)

Collection fields

Element		Type	Description
type	string		REQUIRED. Must be set to Collection to be a valid Collection.
stac_version	string		REQUIRED. The STAC version the Collection implements.
stac_extensions	[string]		A list of extension identifiers the Collection implements.
id	string		REQUIRED. Identifier for the Collection that is unique across all collections in the root catalog.
title	string		A short descriptive one-line title for the Collection.
description	string		REQUIRED. Detailed multi-line description to fully explain the Collection. CommonMark 0.29 syntax MAY be used for rich text representation.
keywords	[string]		List of keywords describing the Collection.
license	string		REQUIRED License(s) of the data collection as SPDX License identifier, SPDX License expression, or other (see below).
providers	[Provider Object]		A list of providers, which may include all organizations capturing or processing the data or the hosting provider.
extent	Extent Object		REQUIRED. Spatial and temporal extents.
summaries	Map<string, [*] Range Object JSON Schema Object >		STRONGLY RECOMMENDED. A map of property summaries, either a set of values, a range of values or a JSON Schema .

Element	Type	Description
links	[Link Object]	REQUIRED. A list of references to other documents.
assets	Map<string, Asset Object >	Dictionary of asset objects that can be downloaded, each with a unique key.
item_assets	Map<string, Item Asset Definition Object >	A dictionary of assets that can be found in member Items.

stac_version

In general, STAC versions can be mixed, but please keep the [recommended best practices](#) in mind.

stac_extensions

A list of extensions the Collection implements. The list consists of URLs to JSON Schema files that can be used for validation. This list must only contain extensions that extend the Collection specification itself, see the ‘Scope’ for each of the extensions. This must **not** declare the extensions that are only implemented in child Collection objects or child Item objects.

id

It is important that Collection identifiers are unique across all collections in the corresponding root catalog. Providers should strive as much as possible to make their Collection ids ‘globally’ unique, prefixing any common information with a unique string. This could be the provider’s name if it is a fairly unique name, or their name combined with the domain they operate in.

license

License(s) of the data that the STAC Collection and its children provides. If possible, license information should be defined at the Collection level.

The license(s) can be provided as:

1. [SPDX License identifier](#)
2. [SPDX License expression](#)
3. String with the value other if the license is not on the SPDX license list. The strings various and proprietary are **deprecated**.

If the license is **not** an SPDX license identifier, links to the license texts SHOULD be added. The links MUST use the [license link relation type](#). If there is no public license URL available, it is RECOMMENDED to supplement the STAC Item with the license text in a separate file and link to this file. If no link to a license is included and the license field is set to other (or one of the deprecated values), the Collection is private, and consumers have not been granted any explicit right to use the data.

providers

A list of providers, which may include all organizations capturing or processing the data or the hosting provider. Providers should be listed in chronological order with the most recent provider being the last element of the list.

Provider Object

The object provides information about a provider. A provider is any of the organizations that captures or processes the content of the Collection and therefore influences the data offered by this Collection. May also include information about the final storage provider hosting the data.

Field Name	Type	Description
name	string	REQUIRED. The name of the organization or the individual.
description	string	Multi-line description to add further provider information such as processing details for processors and producers, hosting details for hosts or basic contact information. CommonMark 0.29 syntax MAY be used for rich text representation.
roles	[string]	Roles of the provider. Any of <code>licensor</code> , <code>producer</code> , <code>processor</code> or <code>host</code> .
url	string	Homepage on which the provider describes the dataset and publishes contact information.

roles: The provider's role(s) can be one or more of the following elements:

- *licensor*: The organization that is licensing the dataset under the license specified in the Collection's license field.
- *producer*: The producer of the data is the provider that initially captured and processed the source data, e.g. ESA for Sentinel-2 data.
- *processor*: A processor is any provider who processed data to a derived product.
- *host*: The host is the actual provider offering the data on their storage. There should be no more than one host, specified as last element of the list.

extents

Extent Object

The object describes the spatio-temporal extents of the Collection. Both spatial and temporal extents are required to be specified.

Element	Type	Description
spatial	Spatial Extent Object	REQUIRED. Potential <i>spatial extents</i> covered by the Collection.
temporal	Temporal Extent Object	REQUIRED. Potential <i>temporal extents</i> covered by the Collection.

Spatial Extent Object

The object describes the spatial extents of the Collection.

Element	Type	Description
bbox	[[number]]	REQUIRED. Potential <i>spatial extents</i> covered by the Collection.

bbox: Each outer array element can be a separate spatial extent describing the bounding boxes of the assets represented by this Collection using either 2D or 3D geometries.

The first bounding box always describes the overall spatial extent of the data. All subsequent bounding boxes can be used to provide a more precise description of the extent and identify clusters of data. Clients only interested in the overall spatial extent will only need to access the first item in each array. It is recommended to only use multiple bounding boxes if a union of them would then include a large uncovered area (e.g. the union of Germany and Chile). Thus, it doesn't make sense to provide two bounding boxes and the validation will fail in this case.

The length of the inner array must be $2 \times n$ where n is the number of dimensions. The array contains all axes of the southwesterly most extent followed by all axes of the northeasterly most extent specified in Longitude/Latitude or Longitude/Latitude/Elevation based on [WGS 84](#). When using 3D geometries, the elevation of the southwesterly most extent is the minimum depth/height in meters and the elevation of the northeasterly most extent is the maximum.

The coordinate reference system of the values is WGS 84 longitude/latitude. Example that covers the whole Earth: `[[-180.0, -90.0, 180.0, 90.0]]`. Example that covers the whole earth with a depth of 100 meters to a height of 150 meters: `[[-180.0, -90.0, -100.0, 180.0, 90.0, 150.0]]`.

Temporal Extent Object

The object describes the temporal extents of the Collection.

Element	Type	Description
interval	<code>[[string null]]</code>	REQUIRED. Potential <i>temporal extents</i> covered by the Collection.

interval: Each outer array element can be a separate temporal extent. The first time interval always describes the overall temporal extent of the data. All subsequent time intervals can be used to provide a more precise description of the extent and identify clusters of data. Clients only interested in the overall extent will only need to access the first item in each array. It is recommended to only use multiple temporal extents if a union of them would then include a large uncovered time span (e.g. only having data for the years 2000, 2010 and 2020).

Each inner array consists of exactly two elements, either a timestamp or `null`.

Timestamps consist of a date and time in UTC and **MUST** be formatted according to [RFC 3339, section 5.6](#). The temporal reference system is the Gregorian calendar.

Open date ranges are supported by setting the start and/or the end time to `null`. Example for data from the beginning of 2019 until now: `[["2019-01-01T00:00:00Z", null]]`. It is recommended to provide at least a rough guideline on the temporal extent and thus it's not recommended to set both start and end time to `null`. Nevertheless, this is possible if there's a strong use case for an open date range to both sides.

summaries

Collections are *strongly recommended* to provide summaries of the values of fields that they can expect from the properties of STAC Items contained in this Collection. This enables users to get a good sense of what the ranges and potential values of different fields in the Collection are, without having to inspect a number of Items (or crawl them exhaustively to get a definitive answer). Summaries are often used to give users a sense of the data in [Standalone Collections](#), describing the potential values even when it can't be accessed as Items. They also give clients enough information to build tailored user interfaces for querying the data, by presenting the potential values that are available. Fields selected to be included in summaries should capture all the potential values of the field that appear in every Item underneath the collection, including in any

nested sub-Catalogs.

A summary for a field can be specified in three ways:

1. A set of all distinct values in an array: The set of values must contain at least one element and it is strongly recommended to list all values. If the field summarizes an array (e.g. [instruments](#)), the field's array elements of each Item must be merged to a single array with unique elements.
2. A Range in a [Range Object](#): Statistics by default only specify the range (minimum and maximum values), but can optionally be accompanied by additional statistical values. The range specified by the minimum and maximum properties can specify the potential range of values, but it is recommended to be as precise as possible.
3. Extensible JSON Schema definitions for fine-grained information, see the [JSON Schema Object](#) section for more.

All values must follow the schema of the property field they summarize, unless the field is an array as described in (1) above. So the values in the array or the values given for minimum and maximum must comply to the original data type and any further restrictions that apply for the property they summarize. For example, the minimum for `gsd` can't be lower than zero and the summaries for `platform` and `instruments` must each be an array of strings (or alternatively minimum and maximum values, but that's not very meaningful).

It is recommended to list as many properties as reasonable so that consumers get a full overview about the properties included in the Items. Nevertheless, it is not very useful to list all potential `title` values of the Items. Also, a range for the `datetime` property may be better suited to be included in the STAC Collection's `extent` field. In general, properties that are covered by the Collection specification should not be repeated in the summaries.

See the [examples folder](#) for Collections with summaries to get a sense of how to use them.

Range Object

For summaries that would normally consist of a lot of continuous values, statistics can be added instead. By default, only ranges with a minimum and a maximum value can be specified. Ranges can be specified for [ordinal](#) values only, which means they need to have a rank order. Therefore, ranges can only be specified for numbers and some special types of strings. Examples: grades (A to F), dates or times. Implementors are free to add other derived statistical values to the object, for example `mean` or `stddev`.

Field Name	Type	Description
minimum	number string	REQUIRED. Minimum value.
maximum	number string	REQUIRED. Maximum value.

JSON Schema Object

For a full understanding of the summarized field, a JSON Schema can be added for each summarized field. This allows very fine-grained information for each field and each value as JSON Schema is also extensible. Each schema must be valid against all corresponding values available for the property in the sub-Items. Empty schemas are not allowed.

JSON Schema draft-07 is the default JSON Schema version, which aligns with the JSON Schemas provided by STAC. It is allowed to use other versions of JSON Schema if the version is explicitly expressed in the JSON Schema `$schema` keyword, but tooling may not support JSON Schema versions other than `draft-07`.

For an introduction to JSON Schema, see “[Learn JSON Schema](#)”.

links

This object is described in the [Links](#) document.

Relation types

All the [common relation types](#) can be used in Collection. A `self` and a `root` links are **STRONGLY RECOMMENDED**. Non-root Collections **SHOULD** include a parent link to their parent.

[!NOTE] The STAC Catalog specification requires a link to at least one `item` or `child` Catalog. This is *not* a requirement for Collections, but *recommended*. In contrast to Catalogs, it is **REQUIRED** that Items linked from a Collection **MUST** refer back to its Collection with a link with the [collection relation type](#).

assets

Collection Assets provides an optional mechanism to expose assets that don’t make sense at the Item level. The property `assets` is a dictionary of [Asset Objects](#), each with a unique key. Each asset refers to data associated with the Collection that can be downloaded or streamed. This construct is further detailed in the [Assets](#) document.

There are a few guidelines for using the asset construct at the Collection level:

- Collection-level assets **SHOULD NOT** list any files also available in Items.
- If possible, item-level assets are always the preferable way to expose assets.

Collection-level assets can be useful in some scenarios, for example:

1. Exposing additional data that applies Collection-wide and you don’t want to expose it in each Item. This can be Collection-level metadata or a thumbnail for visualization purposes.
2. Individual Items can’t properly be distinguished for some data structures, e.g. [Zarr](#) as it’s a data structure not contained in single files.
3. Exposing assets for “[Standalone Collections](#)”.

Often, it is possible to model data and assets with either a Collection or an Item. In those scenarios we *recommend* to use Items as much as possible, as they are designed for assets. Using Collection-level assets should only be used if there is no other option.

item_assets

This serves two purposes:

1. Provide a human-readable definition of assets available in **any** Items belonging to this Collection so that the user can determine the key(s) of assets they are interested in.
2. Provide a way to programmatically determine what assets are available in **any** member Item. Otherwise a random Item needs to be examined to determine assets available, but a random Item may not be representative of the set.

An Item Asset Object defined at the Collection level is nearly the same as the [Asset Object in Items](#),

except for two differences. The `href` field is not required, because Item Asset Definitions don't point to any data by themselves, but at least two other fields must be present.

Item Asset Definition Object

An item asset is an object that contains details about the datafiles that will be included in member Items. Assets included at the Collection level do not imply that all assets are available from all Items. However, it is recommended that the Asset Definition is a complete set of **all** assets that may be available from **any** member Items. So this should be the union of the available assets, not just the intersection of the available assets.

Field Name	Type	Description
title	string	The displayed title for clients and users.
description	string	A description of the Asset providing additional details, such as how it was processed or created. CommonMark 0.29 syntax MAY be used for rich text representation.
type	string	Media type of the asset.
roles	[string]	The semantic roles of the asset, similar to the use of <code>rel</code> in links.

Other custom fields, or fields from other extensions may also be included in the Asset object.

Any property that exists for a Collection-level asset object must also exist in the corresponding assets object in each Item. If a collection's asset object contains properties that are not explicitly stated in the Item's asset object then that property does not apply to the item's asset. Item asset objects at the Collection-level can describe any of the properties of an asset, but those assets properties and values must also reside in the item's asset object. To consolidate item-level asset object properties in an API setting, consider storing the STAC Item objects without the larger properties internally as 'invalid' STAC items, and merge in the desired properties at serving time from the Collection-level.

At least two fields (e.g. `title` and `type`) are required to be provided, in order for it to adequately describe Item assets. The two fields must not necessarily be taken from the list above and may include any custom field.

Media Type for STAC Collections

A STAC Collection is a JSON file ([RFC 8259](#)), and thus should use the [application/json](#) as the [Media Type](#) (previously known as the MIME Type).

Standalone Collections

STAC Collections which don't link to any Item are called **standalone Collections**. To describe them with more fields than the Collection fields has to offer, it is allowed to re-use the metadata fields defined by extensions for Items in the `summaries` field. This makes much sense for fields such as `platform` or `proj:code`, which are often the same for a whole Collection, but doesn't make much sense for `eo:cloud_cover`, which usually varies heavily across a Collection. The data provider is free to decide, which fields are reasonable to be used.

Extensions

STAC Collections are [extensible](#) . Please refer to the [extensions overview](#) to find relevant extensions for STAC Collections.

STAC Commons

This folder contains parts of the specification that are shared across the Item, Catalog and Collection specifications.

In this directory

- [Assets](#) : Asset refers to data associated with Items or Collections that can be downloaded or streamed.
- [Links](#) : Links describe relationships with other entities, e.g. other STAC entities or external resources.
- [Common Metadata](#) : Common metadata defines commonly used fields in STAC that can be used in all STAC entities, e.g. Items, Collections, Catalogs, Assets, Links, Bands, etc.

Note: For backward compatibility, the JSON Schemas for common metadata are located in the [folder that contains the JSON Schemas for the Item specification](#) .

Assets

- [Asset Object](#)
- [Media Types](#)
- [Roles](#)
- [Additional Fields](#)

The property `assets` is a dictionary of [Asset Objects](#), each with a unique key. Each asset refers to data associated with the Item or Collection that can be downloaded or streamed. In general, the keys don't have any meaning and are considered to be non-descriptive unique identifiers. Providers may assign any meaning to the keys for their respective use cases, but must not expect that clients understand them. To communicate the purpose of an asset better use the [roles field](#) in the [Asset Object](#).

Asset Object

An Asset is an object that contains a URI to data associated with the Item that can be downloaded or streamed. It is allowed to add additional fields.

Field Name	Type	Description
<code>href</code>	string	REQUIRED. URI to the asset object. Relative and absolute URI are both allowed. Trailing slashes are significant.
<code>title</code>	string	The displayed title for clients and users.
<code>description</code>	string	A description of the Asset providing additional details, such as how it was processed or created. CommonMark 0.29 syntax MAY be used for rich text representation.

Field Name	Type	Description
type	string	Media type of the asset. See the common media types in the best practice doc for commonly used asset types.
roles	[string]	The semantic roles of the asset, similar to the use of <code>rel</code> in links.

[Additional fields](#) may be added to the assets, though this is recommended only in special cases.

Media Types

Any media type can be used in an Item's asset type field, and [registered](#) Media Types are preferred. STAC Items that have sidecar metadata files associated with a data asset (e.g., `.tifw`, Landsat 8 MTL files) should use media types appropriate for the metadata file. For example, if it is a plain text file, then `text/plain` would be appropriate; if it is an XML, then `text/xml` is appropriate. For more information on media types as well as a list of [common media types](#) used in STAC see the [best practice on working with media types](#).

Roles

The `roles` field is used to describe the purpose of each asset. It is recommended to include one for every asset, to give users a sense of why they might want to make use of the asset. There are some emerging standards that enable clients to take particular action when they encounter particular roles, listed below. But implementors are encouraged to come up with their own terms to describe the role.

Like the `rel` field in Link Objects, the `roles` field can be given any value. However, there are a few standardized role names that can be found in the [best practices](#). Commonly used are `thumbnail`, `overview`, `data` and `metadata`.

Note that multiple roles per asset are encouraged: pick all the ones that apply. So many should have the `data` role, and then another role to describe how the data is used. For more information on how to use roles see the [Asset Roles](#) section of the Best Practices document.

Additional Fields

As detailed above, Items contain properties, which are the main source of metadata for searching across Items. Many content extensions can add further property fields as well. Any property that can be specified for an Item can also be specified for a specific asset. This can be used to override a property defined in the Item, or to specify fields for which there is no single value for all assets. If a property is defined in the Item Properties, it is the default value for all assets in the Item. If a property is defined in a Collection on the top-level, it is the default value for all assets in the Collection. Assets may override the properties inherited for specific assets (see example below). Inheritance does not apply across multiple files, i.e. is restricted to a single Item or Collection. Note that in some cases the inheritance may not lead to the expected results if other semantics have been defined for the property. See the examples for `gsd` and `eo:bands` below. It also applies to some other fields such as `created` or `updated`. Inheritance may be undesirable in these cases, e.g. the `created` attribute of an item would describe when the metadata was generated, whereas on an asset the `created` attribute would describe when the data file itself was created.

It is important to note that the STAC API does not facilitate searching across Asset properties in this way, and this should be used sparingly. It is primarily used to define properties at the Asset level that may be used during use of the data instead of for searching.

For example, `gsd` defined for an Item represents the best Ground Sample Distance (resolution) for the data within the Item. However, some assets may be lower resolution and thus have a higher `gsd`. The `bands` field in combination with the EO extension defines an array of spectral bands. However, it may be useful instead to specify the bands that are used in a particular asset.

For an example see the [sentinel2-sample](#) . The Sentinel-2 overall `gsd` is 10m, because this is the best spatial resolution among all the bands and is defined in Item properties so it can be searched on. In the example Band 5 and others have a `gsd` of 20m, so that asset specifies the `gsd` as well, which overrides the Item `gsd` for this one asset. The example also includes reduced resolution versions of files included as assets, using `gsd` to represent the proper resolution.

For `bands`, it could be put in Item properties as an array of all the bands, but in this case it's not. Instead, the assets each define an array containing the spectral band information for that asset (in the order the bands appear in the file).

For examples of fields that this construct is recommended for, see the [section of STAC Best Practices](#) that talks about common use cases of additional fields for assets.

Links

- [Link Object](#)
 - [HTTP headers](#)
- [Media Types](#)
 - [STAC Media Types](#)
- [Relation Types](#)
 - [Hierarchical relations](#)
 - [self relation](#)
 - [root and parent relation](#)
 - [child relation](#)
 - [collection and item relation](#)

Link Object

This object describes a relationship with another entity. Data providers are advised to be liberal with the links section, to describe things like the Catalog an Item is in, related Items, parent or child Items (modeled in different ways, like an 'acquisition' or derived data).

Field Name	Type	Description
<code>href</code>	string	REQUIRED. The actual link in the format of an URL. Relative and absolute links are both allowed. Trailing slashes are significant.
<code>rel</code>	string	REQUIRED. Relationship between the current document and the linked document. See chapter "Relation types" for more information.
<code>type</code>	string	Media type of the referenced entity.
<code>title</code>	string	A human readable title to be used in rendered displays of the link.
<code>method</code>	string	The HTTP method that shall be used for the request to the target resource, in uppercase. GET by default
<code>headers</code>	Map<string, string	The HTTP headers to be sent for the request to the target resource.

Field Name	Type	Description
	[string]>	
body	any	The HTTP body to be sent to the target resource.

For a full discussion of the situations where relative and absolute links are recommended see the [‘Use of links’](#) section of the STAC best practices.

HTTP headers

The field `headers` allows to describe a dictionary of HTTP headers that are required to be sent by the client. The keys of the dictionary are the header names, and the values are either a single string or an array of strings. In case of an array, the header is expected to be sent multiple times with the different values.

Media Types

One of the best ways to help inform web clients about the content in a link is to use a common [Media Type](#) in the `type` field. In STAC the `type` field is used in both the [Link Object](#) and in the [Asset Object](#) . It is quite useful for STAC browsers to better determine what to render and display to users searching and browsing the Catalog. Media types are sometimes referred to by the now deprecated term “MIME types”.

STAC Media Types

The following table lists the Media Types to use for STAC entities.

Media Type	Description
application/geo+json	A STAC Item
application/json	A STAC Catalog
application/json	A STAC Collection

Relation Types

STAC Entities use a variety of `rel` types in the Link Object, to describe the exact nature of the link between the STAC object and the entity it is linking to. It is recommended to use the official [IANA Link Relation Types](#) where possible.

Hierarchical relations

The following table lists the STAC-specific `rel` types that are used in the `links` object of a STAC entity to link with other STAC entities in the same catalog.

Type	Description	Media Type
self	<i>Absolute</i> URL to the location that the STAC file can be found online, if available.	application/json
root	URL to the root STAC entity (Catalog or Collection).	application/json
parent	URL to the parent STAC entity (Catalog or Collection).	application/json
child	URL to a child STAC entity (Catalog or Collection).	application/json

Type	Description	Media Type
collection	URL to the parent Collection. <i>Absolute</i> URLs should be used whenever possible.	application/json
item	URL to a STAC Item.	application/geo+json (preferred) or application/json

self relation

The `self` relation is used to link to the STAC entity itself. This is particularly useful when in a download package that includes metadata, so that the downstream user can know where the data has come from.

root and parent relation

The `root` and `parent` relations are used to link to the root and parent STAC entity, which is either a [Catalog](#) or a [Collection](#). Conceptually, STAC entities SHALL have no more than one parent entity. As such, STAC entities also can have no more than one root entity. Therefore, there's usually just one link with `root` or `parent` relationship unless different variations of the same conceptual entity exist (identified by the ID). Different variations could be:

- a different encoding (see the `type` property), e.g. a HTML version in addition to JSON
- a different language (see the `hreflang` property). e.g. a German version in addition to English

child relation

The `child` relation is **ONLY** used to link a catalog or collection to a child catalog or collection.

collection and item relation

The `collection` and `item` relations are used to link to the parent collection and a child Item, respectively. It is **RECOMMENDED** to link an `item` from a collection and not directly from a catalog. All Items linked from a Collection **MUST** refer back to its Collection with the `collection` relation type. The referenced Collection is **STRONGLY RECOMMENDED** to implement the same STAC version as the Collection.

STAC Common Metadata

This document outlines commonly used fields in STAC. They are often used in [STAC Item properties](#), but can also be used in other places, e.g. an [Item Asset](#) or [Collection Asset](#).

- [Basics](#)
- [Date and Time](#)
 - [Date and Time Range](#)
- [Licensing](#)
 - [License relation](#)
- [Provider](#)
 - [Provider Object](#)
 - [roles](#)
- [Instrument](#)
 - [platform](#)
 - [instruments](#)

- [constellation](#)
 - [mission](#)
 - [gsd](#)
- [Bands](#)
 - [Band Object](#)
- [Data Values](#)
 - [No-data](#)
 - [Units](#)
 - [Statistics Object](#)
 - [Data Types](#)

Various *examples* are available in the folder [examples](#) . *JSON Schemas* can be found in the folder [json-schema](#) .

Implementation of any of the fields is not required, unless explicitly required by a specification using the field. For example, `datetime` is required in STAC Items.

Basics

Descriptive fields to give a basic overview of a STAC entity (e.g. Catalog, Collection, Item, Asset).

- [JSON Schema](#)

Field Name	Type	Description
title	string	A human readable title describing the STAC entity.
description	string	Detailed multi-line description to fully explain the STAC entity. CommonMark 0.29 syntax MAY be used for rich text representation.
keywords	[string]	List of keywords describing the STAC entity.
roles	[string]	The semantic roles of the entity, e.g. for assets, links, providers, bands, etc.

Date and Time

- [JSON Schema](#)

Fields to provide additional temporal information such as ranges with a start and an end datetime stamp.

Field Name	Type	Description
datetime	string null	See the Item Specification Fields for more information.
created	string	Creation date and time of the corresponding STAC entity or Asset (see below), in UTC.
updated	string	Date and time the corresponding STAC entity or Asset (see below) was updated last, in UTC.

All timestamps MUST be formatted according to [RFC 3339, section 5.6](#) .

created and **updated** have different meaning depending on where they are used. If those fields are available in a Collection, in a Catalog (both top-level), or in a Item (in the `properties`), the fields refer the metadata (e.g., when the STAC metadata was created). Having those fields in the Assets or Links, they refer to the actual data linked to (e.g., when the asset was created).

NOTE: There are more date and time related fields available in the [Timestamps extension](#) .

Date and Time Range

While a STAC entity (e.g. an Item) can have a nominal datetime describing the capture, these properties allow a STAC entity to have a range of capture dates and times. An example of this is the [MODIS 16 day vegetation index product](#) .

Important: Using one of the fields REQUIRES inclusion of the other field as well to enable a user to search STAC records by the provided times. So if you use `start_datetime` you need to add `end_datetime` and vice-versa. Both fields are also REQUIRED if the `datetime` field is set to `null`. The datetime property in a STAC Item and these fields are not mutually exclusive.

Field Name	Type	Description
<code>start_datetime</code>	string	The first or start date and time for the resource, in UTC. It is formatted as date-time according to RFC 3339, section 5.6 .
<code>end_datetime</code>	string	The last or end date and time for the resource, in UTC. It is formatted as date-time according to RFC 3339, section 5.6 .

`start_datetime` and `end_datetime` constitute inclusive bounds, meaning that the range covers the entire time interval between the two timestamps and the timestamps itself.

Licensing

Information about the license(s) of the data, which is not necessarily the same license that applies to the metadata. **Licensing information should be defined at the [Collection](#) level if possible.**

- [JSON Schema](#)

Field Name	Type	Description
<code>license</code>	string	License(s) of the data as SPDX License identifier, SPDX License expression, or other (see below).

license: License(s) of the data that the STAC entity provides.

The license(s) can be provided as:

1. [SPDX License identifier](#)
2. [SPDX License expression](#)
3. String with the value `other` if the license is not on the SPDX license list. The strings `various` and `proprietary` are **deprecated**.

If the license is **not** an SPDX license identifier, links to the license texts SHOULD be added. The links MUST use the [license link relation type](#) . If there is no public license URL available, it is RECOMMENDED to supplement the STAC Item with the license text in a separate file and link to this file. If no link to a license is included and the `license` field is set to `other` (or one of the deprecated values), the data is private, and consumers have not been granted any explicit right to use it.

License relation

Type	Description
license	The license URL(s) for the resource SHOULD be specified if the <code>license</code> field is not a SPDX license identifier.

Provider

Information about the organizations capturing, producing, processing, hosting or publishing this data.

Provider information should be defined at the Collection level if possible.

- [JSON Schema](#)

Field Name	Type	Description
providers	Provider Object	A list of providers, which may include all organizations capturing or processing the data or the hosting provider. Providers should be listed in chronological order with the most recent provider being the last element of the list.

Provider Object

The object provides information about a provider. A provider is any of the organizations that captures or processes the content of the assets and therefore influences the data offered by the STAC implementation. May also include information about the final storage provider hosting the data.

Field Name	Type	Description
name	string	REQUIRED. The name of the organization or the individual.
description	string	Multi-line description to add further provider information such as processing details for processors and producers, hosting details for hosts or basic contact information. CommonMark 0.29 syntax MAY be used for rich text representation.
roles	[string]	Roles of the provider. Any of <code>licensor</code> , <code>producer</code> , <code>processor</code> or <code>host</code> .
url	string	Homepage on which the provider describes the dataset and publishes contact information.

roles

The provider's role(s) can be one or more of the following elements:

- *licensor*: The organization that is licensing the dataset under the license specified in the Collection's `license` field.
- *producer*: The producer of the data is the provider that initially captured and processed the source data, e.g. ESA for Sentinel-2 data.
- *processor*: A processor is any provider who processed data to a derived product.
- *host*: The host is the actual provider offering the data on their storage. There should be no more than one host, specified as the last element of the provider list.

Instrument

Adds metadata specifying a platform and instrument used in a data collection mission. These fields will often

be combined with domain-specific extensions that describe the actual data, such as the `eo` or `sar` extensions.

- [JSON Schema](#)

Field Name	Type	Description
<code>platform</code>	string	Unique name of the specific platform to which the instrument is attached.
<code>instruments</code>	[string]	Name of instrument or sensor used (e.g., MODIS, ASTER, OLI, Canon F-1).
<code>constellation</code>	string	Name of the constellation to which the platform belongs.
<code>mission</code>	string	Name of the mission for which data is collected.
<code>gsd</code>	number	Ground Sample Distance at the sensor, in meters (m), must be greater than 0.

platform

The unique name of the specific platform the instrument is attached to. For satellites this would be the name of the satellite, whereas for drones this would be a unique name for the drone. Examples include `landsat-8` (Landsat-8), `sentinel-2a` and `sentinel-2b` (Sentinel-2), `terra` and `aqua` (part of NASA EOS, carrying the MODIS instruments), `mycorp-uav-034` (hypothetical drone name), and `worldview02` (Maxar/DigitalGlobe WorldView-2).

instruments

An array of all the sensors used in the creation of the data. For example, data from the Landsat-8 platform is collected with the OLI sensor as well as the TIRS sensor, but the data is distributed together so would be specified as `['oli', 'tirs']`. Other instrument examples include `msi` (Sentinel-2), `aster` (Terra), and `modis` (Terra and Aqua), `c-sar` (Sentinel-1) and `asar` (Envisat).

constellation

The name of a logical collection of one or more platforms that have similar payloads and have their orbits arranged in a way to increase the temporal resolution of acquisitions of data with similar geometric and radiometric characteristics. This field allows users to search for related data sets without the need to specify which specific platform the data came from, for example, from either of the Sentinel-2 satellites. Examples include `landsat-8` (Landsat-8, a constellation consisting of a single platform), `sentinel-2` ([Sentinel-2](#)), `rapideye` (operated by Planet Labs), and `modis` (NASA EOS satellites Aqua and Terra). In the case of `modis`, this is technically referring to a pair of sensors on two different satellites, whose data is combined into a series of related products. Additionally, the Aqua satellite is technically part of the A-Train constellation and Terra is not part of a constellation, but these are combined to form the logical collection referred to as MODIS.

mission

The name of the mission or campaign for collecting data. This could be a discrete set of data collections over a period of time (such as collecting drone imagery), or could be a set of tasks of related tasks from a satellite data collection.

gsd

The nominal Ground Sample Distance for the data, as measured in meters on the ground. There are many

definitions of GSD. The value of this field should be related to the spatial resolution at the sensor, rather than the pixel size of images after orthorectification, pansharpening, or scaling. The GSD of a sensor can vary depending on geometry (off-nadir / grazing angle) and wavelength, so it is at the discretion of the implementer to decide which value most accurately represents the GSD. For example, Landsat8 optical and short-wave IR bands are all 30 meters, but the panchromatic band is 15 meters. The `gsd` should be 30 meters in this case because that is the nominal spatial resolution at the sensor. The Planet PlanetScope Ortho Tile Product has an `gsd` of 3.7 (or 4 if rounding), even though the pixel size of the images is 3.125. For example, one might choose for WorldView-2 the Multispectral 20° off-nadir value of 2.07 and for WorldView-3 the Multispectral 20° off-nadir value of 1.38.

Bands

Field Name	Type	Description
<code>bands</code>	[Band Object]	An array of available bands where each object is a Band Object .

The `bands` array is used to describe the available bands in a STAC entity or Asset. This field describes the general construct of a band or layer, which doesn't necessarily need to be a spectral band. By adding fields from extensions you can indicate that a band, for example, is

- a spectral band ([EO extension](#)),
- a band with classification results ([classification extension](#)),
- a band with quality information such as cloud cover probabilities,
- etc.

Please refer to the [Bands best practices](#) for more details.

[!NOTE] This property is the successor of the `eo:bands` and `raster:bands` fields, which has been present in previous versions of these extensions. The behavior is very similar and they can be migrated easily. Usually, you can simply merge each object on a by-index basis. Nevertheless, you should consider deduplicating properties with the same values across all bands to the asset level (see the [best practices](#)). For some fields, you need to add the extension prefix of the `eo` or `raster` extension to the property name though. See the [Band migration best practice](#) for details.

Band Object

Specifically defined for the Band Object is just a single property name, which serves as a unique identifier. You can add additional fields from the common metadata such as a [description](#) or the [value-related](#) properties.

Field Name	Type	Description
<code>name</code>	string	The name of the band (e.g., “B01”, “B8”, “band2”, “red”), which should be unique across all bands defined in the list of bands. This is typically the name the data provider uses for the band.
<code>description</code>	string	Description to fully explain the band. CommonMark 0.29 syntax MAY be used for rich text representation.

A Band Object must contain at least one property, which is not necessarily one of the properties defined here

and can be a property from an extension or common metadata.

Data Values

Adds metadata about the data values or measurement values contained in the entity that is described by the object these fields get added to (e.g., an asset or a band). These fields will often be combined with extensions that group data values into a “unit” or “chunk”, e.g., a band or layer in a file (raster and eo extensions), a column in a table (table extension), or dimensions in a datacube (datacube extension).

Field Name	Type	Description
nodata	number string	Value used to identify no-data, see below .
data_type	string	The data type of the values, see below .
statistics	Statistics Object	Statistics of all the values.
unit	string	Unit of measurement of the value, see below .

No-data

The no-data value must be provided either as:

- a number
- a string:
 - nan - [NaN](#) (not a number) as defined in IEEE-754
 - inf - Positive Infinity
 - -inf - Negative Infinity

Units

It is STRONGLY RECOMMENDED to provide units in one of the following two formats:

- [UCUM](#) : The unit code that is compliant to the UCUM specification.
- [UDUNITS-2](#) : The unit symbol if available, otherwise the singular unit name.

Statistics Object

Statistics usually specify the range of values by providing the minimum and maximum values, but can optionally be accompanied by additional statistical values. Some additional statistical sizes are listed below, but the object can also be extended with other statistical sizes that are not listed below. For example, it could list additional coverages such as vegetation cover, land cover, etc. If statistics are provided in the Item Properties ([example](#)), it is recommended to list the statistical sizes with a JSON Schema in the Collection Summaries to better describe the sizes ([example](#)). Please note that some statistical sizes such as cloud cover have explicit fields in other extensions such as the EO extension. It is recommended to use the fields standardized in extensions in favor of providing them in the Statistics Object.

Field Name	Type	Description
minimum	number	minimum value of the values in the band. If not present, the minimum value of the given data type or negative infinity can be assumed.
maximum	number	maximum value of the values in the band. If not present, the maximum value of the given data type or positive infinity can be assumed.

Field Name	Type	Description
mean	number	mean value of all the values in the band
stddev	number	standard deviation value of the values in the band
count	integer	Total number of all data values (≥ 0)
valid_percent	number	Percentage of valid (not nodata) values (0-100)

Data Types

The data type gives information about the values. This can be used to indicate the (maximum) range of numerical values expected. For example `uint8` indicates that the numbers are in a range between 0 and 255, they can never be smaller or larger. This can help to pick the optimal numerical data type when reading the files to keep memory consumption low. Nevertheless, it doesn't necessarily mean that the expected values fill the whole range. For example, there can be use cases for `uint8` that just use the numbers 0 to 10 for example. Through the [Statistics Object](#) it is possible to specify an exact value range so that visualizations can be optimized. The allowed values for `data_type` are:

- `int8`: 8-bit integer
- `int16`: 16-bit integer
- `int32`: 32-bit integer
- `int64`: 64-bit integer
- `uint8`: unsigned 8-bit integer (common for 8-bit RGB PNG's)
- `uint16`: unsigned 16-bit integer
- `uint32`: unsigned 32-bit integer
- `uint64`: unsigned 64-bit integer
- `float16`: 16-bit float
- `float32`: 32-bit float
- `float64`: 64-bit float
- `cint16`: 16-bit complex integer
- `cint32`: 32-bit complex integer
- `cfloat32`: 32-bit complex float
- `cfloat64`: 64-bit complex float
- `other`: Other data type than the ones listed above (e.g. boolean, string, higher precision numbers)

Extensions

- [Extensions](#)
 - [Overview](#)
 - [Using Extensions](#)
 - [Extension identifiers in `stac\_extensions`](#)
 - [Community Extensions](#)
 - [Proposed extensions](#)
 - [Extension Maturity](#)
 - [Extending STAC](#)
 - [General Conventions](#)
 - [Proposing new extensions](#)
 - [Prefixes](#)
 - [Use of arrays and objects](#)

Overview

One of the most important aspects of the SpatioTemporal Asset Catalog specification is its extensibility. The core STAC specification defines only a minimal core, but is designed for extension. It is expected that most real-world implementations will use several ‘extensions’ to fully describe their data. This document describes how extensions work.

For a list of most available extensions see the [STAC extensions overview page](#) . Please note the [extension maturity](#) for each extension.

Extensions to the core STAC specification provide additional fields that can be used to better describe the data. Most tend to be about describing a particular domain or type of data, but some imply functionality.

Extensions include a JSON Schema precisely describing the structure, a natural language description of the fields, and thorough examples.

Anybody can create an extension for their data, and data providers often work together to share fields between them to create a shared community extension. See the section below on ‘[Extending STAC](#)’) for information on how to get started. Everyone is encouraged to add their extensions to the [STAC extensions overview page](#) , so others can be aware of it.

Each extension should have at least one *owner*. You can find extension owners in each extension’s README.

Using Extensions

When deciding how to model data in STAC it is highly recommended to first look at the [list of extensions](#) and re-use fields there instead of creating your own version. This increases interoperability, as users know that the meaning of your fields is the same as in other STAC implementations. Many clients will also understand more mature extensions for better display and querying.

To incorporate an extension in STAC the ‘Identifier’ of the extension must be added to the `stac_extensions` array of the STAC [Catalog](#) , [Collection](#) or [Item](#) object. This identifier is a URL to the JSON Schema that allows to validate the fields in the extension. These JSON Schema URLs also include the version number of the extension. The ‘Identifier’ can usually be found in the first lines of the README of any extension made with the [extension template](#) .

Extension identifiers in `stac_extensions`

Generally, if an extension is implemented in a STAC file in a place where the extension scope applies to, the extension identifier should be added to the `stac_extension` array. The scope of an extension is usually explained in the README of an extension. Implementing an extension by following the specified requirements usually means including fields, but occasionally also means implementing alternate behaviors.

There is no direct inheritance between children and parents though, so if for example an Item implements an extension, but the Collection doesn’t reflect the usage of the extension, the extension identifier must only be added to the `stac_extension` array in the Item, but not to the Collection. If the Collection itself implements the extension though or ‘summarizes’ a field in Collection Summaries or Item Asset Definitions, the extension identifier should be added to the Collection.

Examples

- If the Catalog, Collection or Item object directly implements the extension, the `stac_extensions` of that object should contain the extension Identifier.
- If an Asset object implements an extension, the `stac_extensions` of the Item or Collection which holds that Asset should contain the extension identifier.
- If a Collection [summary](#) contains Item fields that implement an extension, then the `stac_extensions` array of that Collection should list the extension identifier. For example, if a Collection `summaries` field contains a summary of `eo:cloud_cover`, then that Collection should have the EO extension JSON Schema URL in the `stac_extensions` array.
- If an object implements an extension that results in fields from a separate extension to be referenced, then the latter extension identifier should be included in the `stac_extensions` array for that object. For example, if a Collection implements the [item_assets](#) extension, and in the `item_assets` field there is an Asset Definition which includes `proj:wkt2`, then the Projection extension identifier should be listed in that Collection's `stac_extensions`.

Community Extensions

Everyone is welcome to contribute extensions to the STAC ecosystem. The center of activity for these is the [stac-extensions GitHub organization](#) , which has a number of extension repositories. Some of these, especially the [stable extensions](#), are observed by the STAC PSC. The community can also host STAC extensions in other places, but we encourage the community to at least list them in the [STAC extensions overview page](#) so that everyone can be aware of all extensions at any time and a high level of interoperability is possible.

Proposed extensions

Beyond the community extensions there have been a number of extensions that people have proposed to the STAC community. These can be found in the STAC [Issue Tracker](#) under the [new extension](#) label. These are ideas that others would likely use and potentially collaborate on. Anyone is free to add new ideas there, and see the section below on [proposing new extensions](#) for the workflow to advance ideas into full-fledged community extensions.

Extension Maturity

There are many extensions being built with STAC, but they have varying degrees of maturity. All community extensions listed here included must include a maturity classification, so that STAC spec users can easily get a sense of how much they can count on the extension.

Maturity Classification	Min Impl #	Description	Stability
Proposal	0	An idea put forward by a community member to gather feedback	Not stable - breaking changes almost guaranteed as implementers try out the idea.
Pilot	1	Idea is fleshed out, with examples and a JSON schema, and implemented in one or more catalogs. Additional implementations encouraged to help give feedback	Approaching stability - breaking changes are not anticipated but can easily come from additional feedback.

Maturity Classification	Min Impl #	Description	Stability
Candidate	3	A number of implementers are using it and are standing behind it as a solid extension. User can generally count on an extension at this maturity level.	Mostly stable, breaking changes require a new version and minor changes are unlikely. The extension has a code owner, designated in its README.
Stable	6	Highest current level of maturity. The community of extension maintainers commits to a STAC review process for any changes, which are not made lightly.	Completely stable, all changes require a new version number and review process.
Deprecated	N/A	A previous extension that has likely been superseded by a newer one or did not work out for some reason.	DO NOT USE, is not supported

Maturity mostly comes through diverse implementations, so the minimum number of implementations column is the main gating function for an extension to mature. But extension authors can also choose to hold back the maturity advancement if they don't feel they are yet ready to commit to the less breaking changes of the next level.

Extending STAC

Anyone is welcome to extend STAC and evolve the additions into a full STAC extension with a README, JSON Schema and examples. There are several types of extensions, some just add additional fields, some change the behavior of STAC and some introduce completely new functionality. New extensions should try to align with existing extensions as well as possible and may even re-use fields and their definitions until they may get split into a new extension that combines commonly used fields across multiple extensions.

General Conventions

Creating a new extension usually involves defining a set of logically grouped fields, and specifying what the allowed values for those fields are. This should be done in the extension text (README) and in JSON Schema, to provide validation. While one can theoretically add fields anywhere in JSON there are some conventions as to where to add them in STAC objects.

1. Additional attributes relating to an [Item](#) should be added into the Item Properties object, rather than directly in the Item object.
2. In general, additional attributes that apply to an Item Asset should also be allowed in Item Properties and vice-versa. For example, the `gsd` attribute may be used in Item Properties to describe the best GSD available in the Item Asset objects contained in the Item, but may also be used in an individual Item Asset to describe only the specific GSD of that asset.
3. Additional attributes relating to a [Catalog](#) or [Collection](#) should be added to the top-level of the object.
4. All other objects can generally also be extended, e.g. Link Objects, Provider Objects, Band Objects, etc.
5. Extensions may also extend other extensions, declaring that dependency in the text and JSON Schema.

Proposing new extensions

Extensions can be hosted anywhere, but should use the [extension template](#) as a starting point. If you'd like to add a repository to the [stac-extensions](#) GitHub organization, just ask on [Gitter](#) ! This is fine for work-in-progress extensions. You can also host the extension repository in your own GitHub account, and optionally transfer it to the stac-extensions organization later.

For new extensions that require community discussion, we recommend the following workflow:

- Use the stac-extensions template to sketch out your proposed extension
- Open an issue on this repository with the prefix “New Extension:” and describe the extension. Include a link to the extension repository. Also post it in the Gitter chat for broader recognition.
- Discussion should take place as issues/pull requests on the extension repository directly, but can also occur on the issue created before.
- Once the extension has an initial release, the issue on stac-spec will be closed.

Prefixes

A STAC Item can combine schema information from several different sources - the core STAC Item information, an earth observation community extension, and a vendor specific provider. It can be difficult to distinguish exactly where each definition came from, and to pull out the most relevant information, especially when vendors often will dump in all the metadata they have in to the STAC definition.

So one idea is to have prefixes to differentiate specific vendors (like dg: for DigitalGlobe), and for communities of practice (like eo: for Electro-Optical). These wouldn't be full namespacing, though an extension for like JSON-LD could potentially evolve to make fully resolved namespacing an option.

An example of this can be seen in a Landsat example:

```
"properties": {
  "datetime": "2018-01-01T13:21:30Z",

  "start_datetime": "2018-01-01T13:21:30Z",
  "end_datetime": "2018-01-01T13:31:30Z",

  "view:off_nadir": -0.001,
  "eo:cloud_cover": 10.31,
  "view:sun_azimuth": 149.01607154,
  "view:sun_elevation": 59.21424700,
  "gsd": 30,

  "l8:data_type": "L1T",
  "l8:wrs_path": 153,
  "l8:wrs_row": 25,
  "l8:earth_sun_distance": 1.0141560,
  "l8:ground_control_points_verify": 114,
  "l8:geometric_rmse_model": 7.562,
  "l8:image_quality_tirs": 9,
  "l8:ground_control_points_model": 313,
  "l8:geometric_rmse_model_x": 5.96,
  "l8:geometric_rmse_model_y": 4.654,
  "l8:geometric_rmse_verify": 5.364,
  "l8:image_quality_oli": 9
}
```

Use of arrays and objects

For extensions, it is recommended to

1. Use arrays only as enumerations/lists (possibly sorted), without implying additional meaning (such as order)
2. To avoid using nested objects, in favor of multiple attributes with a similar naming scheme.

For example, if one would like to define an extension to contain a start and a end date, there are multiple options (tl;dr: option **3** is recommended):

1. Define an object, for example: `"date_range": {"start": "2018-01-01", "end": "2018-01-31"}`. This is **discouraged** as it is more complex to search in objects.
2. Define an two-element array where the first element is the start date and the second element is the end date, for example `"date_range": ["2018-01-01", "2018-01-31"]`. This is **discouraged** as it would conflict with Collection summaries, which always considers arrays as true (potentially sorted) enumeration without any additional meaning.
3. Define two separate fields, e.g. `"date_range_start": "2018-01-01", "date_range_end": "2018-01-31"`. This is **recommended** as it avoids the conflicts above and is usually better displayed in software that only understands GeoJSON but has no clue about STAC. This is due to the fact that most legacy software can not display arrays or objects GeoJSON properties properly.

This rules only applies to the fields defined directly for the Item's properties. For fields and structures defined on other levels (e.g. in the root of an Item or in an array), extension authors can freely define the structure. So an array of objects such as the bands are fine to use, but keep in mind that the drawbacks mentioned above usually still apply.